CYBERSECURITY INTERNSHIP

Week 1 — Security Assessment Report

OWASP Juice Shop Vulnerability Assessment

Intern Name	Parwarsha Shaikh	
DHC ID	DHC-5646	
Department	Cyber Security	
Company	DevelopersHub Corporation	
Report Date	April 28, 2026	
Application Tested	OWASP Juice Shop (localhost:3000)	
Tools Used	Browser Dev Tools, OWASP ZAP 2.17.0, Manual Testing	

1. Executive Summary

This report documents findings from a Week 1 security assessment conducted on the OWASP Juice Shop web application — a deliberately vulnerable application used for cybersecurity training. The assessment involved manual vulnerability testing using browser developer tools and automated scanning using OWASP ZAP 2.17.0.

Total Alerts	Critical	High	Medium
13	**2**	**3**	**8**

2. Manual Testing Findings

The following vulnerabilities were identified through manual testing using browser developer tools:

#	Vulnerability	Location	Severity	Description
1	**SQL Injection**	Login Page	**CRITICAL**	By entering ' OR 1=1-- in the email field, authentication was bypassed and admin access was gained without valid credentials.
2	**Cross-Site Scripting (XSS)**	Search Bar	**HIGH**	Injecting <script>alert('XSS');</script> and iframe payloads into the search field rendered the payload in the page, confirming reflected XSS.
3	**Weak Password Policy**	Registration Page	**MEDIUM**	The app displays password requirements but does not enforce them server-side. Password '12345' was accepted. Google flagged it as found in known data breaches.
4	**Improper Error Handling**	Application-wide	**MEDIUM**	The app exposes raw unhandled error messages, revealing internal application structure and server details to users.

3. OWASP ZAP Automated Scan Findings

OWASP ZAP version 2.17.0 was used to perform an automated scan against http://localhost:3000. The scan completed successfully and identified 13 alerts total.

#	Vulnerability	Severity	Description
1	**SQL Injection**	**CRITICAL**	ZAP confirmed SQL injection in multiple endpoints. Unsanitized input is passed directly to database queries.
2	**Content Security Policy (CSP) Header Not Set**	**HIGH**	No CSP header is set, making the app more vulnerable to XSS and data injection attacks.
3	**Cross-Domain Misconfiguration**	**HIGH**	Overly permissive CORS policy allows requests from unauthorized domains, potentially exposing sensitive data.
4	**Missing Anti-Clickjacking Header**	**MEDIUM**	No X-Frame-Options header is set, making the app vulnerable to clickjacking attacks.
5	**Session ID in URL Rewrite**	**MEDIUM**	Session tokens exposed in URL parameters can be captured in browser history or server logs.
6	**Private IP Disclosure**	**MEDIUM**	Internal IP addresses are exposed in server responses, aiding attackers in mapping the internal network.
7	**Strict-Transport-Security Header Not Set**	**MEDIUM**	HSTS is not configured, allowing users to connect over HTTP and making them vulnerable to man-in-the-middle attacks.
8	**Timestamp Disclosure**	**MEDIUM**	Unix timestamps disclosed in server responses reveal information about application internals.
9	**X-Content-Type-Options Header Missing**	**MEDIUM**	Without this header, browsers may MIME-sniff responses, enabling attacks where malicious files are interpreted differently.

4. Areas of Improvement

- Input Sanitization: All user inputs must be validated and sanitized server-side to prevent SQL injection and XSS. Use parameterized queries for all database operations.
- Password Policy Enforcement: Enforce strong password requirements server-side — minimum 8 characters with uppercase, lowercase, digit, and special character. Never rely only on frontend validation.
- Security Headers: Implement HTTP security headers including Content-Security-Policy, X-Frame-Options, Strict-Transport-Security, and X-Content-Type-Options.
- Error Handling: Implement generic error messages for users and log detailed errors server-side only. Never expose stack traces or internal details to the client.
- CORS Policy: Restrict cross-origin requests to trusted domains only.
- Session Management: Never expose session tokens in URLs. Use secure, HttpOnly cookies for session management.
- HTTPS Enforcement: Enable HTTPS and configure HSTS to prevent downgrade attacks.

5. Conclusion

The security assessment of the OWASP Juice Shop application revealed multiple critical and high severity vulnerabilities. The most serious finding was SQL Injection which allowed full admin access bypass without valid credentials. Cross-Site Scripting was confirmed in the search functionality and multiple missing security headers were identified by OWASP ZAP automated scanning.

These vulnerabilities will be addressed in Week 2 through implementing input sanitization using the validator library, password hashing using bcrypt, token-based authentication using jsonwebtoken, and HTTP security headers using Helmet.js.